

LECTURE 16

Mapping EER Models to Relational Schemas

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The overall mapping process, as an ordered sequence of rules
- Mapping strong entity types, and weak entity types (composite keys)
- Mapping 1:1, 1:N, and M:N relationships
- Mapping multi-valued attributes
- Mapping specialization/generalization — three competing strategies, and how to choose
- Mapping categorization (union types) and aggregation
- Mapping EER constraints into CHECK, NOT NULL, and their real limits

The Mapping Process, Applied in Order

An EER diagram has to be mapped down into relations, primary keys, and foreign keys before it becomes a real schema — in a fixed order, because later rules depend on relations the earlier rules already produced.

1. Strong entity types

One relation per entity type, its identifier as primary key



2. Weak entity types

Composite key: partial key + owner's entire primary key



3. Relationships (1:1, 1:N, M:N)

Foreign key placement, or a new relation for M:N



4. Multi-valued attributes

Always become their own relation



5. Specialization / generalization

Choose a strategy — several are viable



6. Categorization and aggregation

Both reduce to relations already produced by steps 1–5



7. Constraints

NOT NULL, CHECK, and their declarative limits

Rule 1 — Mapping Strong Entity Types

For each strong entity type, create one relation. Every simple attribute becomes a column; the entity's identifier becomes the primary key.

Branch(branchNo, street, city, postcode)

branchNo	street	city	postcode
B003	163 Main St	Lahore	54000

Nothing more is needed — a strong entity type's independent existence and self-contained identifier map directly onto a relation's defining properties.

Rule 2 — Mapping Weak Entity Types

A weak entity type's primary key is NOT just its own partial key — it must include the ENTIRE primary key of its owner, because the weak entity cannot be identified without knowing which owner it belongs to.

Bed(wardNo, bedNo, status) — wardNo is PK component AND FK

wardNo	bedNo	status
W1	1	Occupied
W1	2	Free
W2	1	Occupied

CREATE TABLE Bed

```
CREATE TABLE Bed (  
  wardNo  VARCHAR(4)  NOT NULL,  
  bedNo   SMALLINT    NOT NULL,  
  status  VARCHAR(10) NOT NULL,  
  PRIMARY KEY (wardNo, bedNo),  
  FOREIGN KEY (wardNo) REFERENCES Ward(wardNo) ON DELETE CASCADE  
);
```

ON DELETE CASCADE: Bed is existence-dependent on Ward — deleting a ward should delete its beds automatically.

Rule 3 — Mapping 1:1 Relationships

No new relation is needed — the foreign key goes on ONE of the two existing relations: the side with MANDATORY (total) participation, since that side is guaranteed to always have a value to store.

Department HeadedBy Doctor: every department must have exactly one head (1,1 — total); not every doctor heads a department (0,1 — partial). FK goes on Department.

Department(deptNo, name, location, headDoctorNo)

deptNo	name	location	headDoctorNo
D01	Cardiology	Block C	S014

When both sides are total

The foreign key can go on EITHER side — or the two relations can simply be merged into one, since a strict 1:1 mandatory-both-sides relationship means the two entity types always co-occur in lockstep. Which relation gets the FK is then a matter of convenience, not correctness.

Rule 4 — Mapping 1:N Relationships

The foreign key always goes on the relation representing the MANY side, referencing the primary key of the relation on the ONE side. No new relation is created.

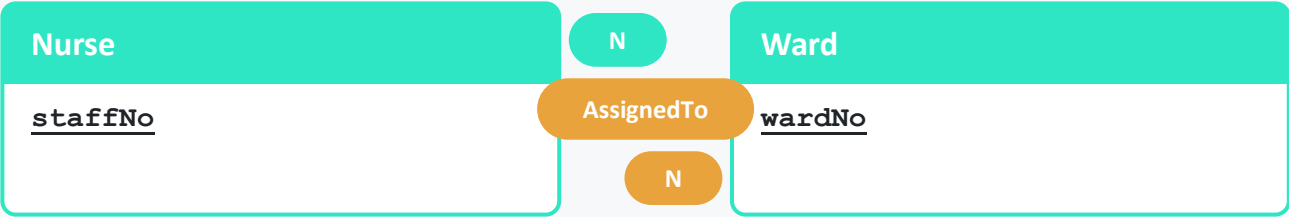
`Staff(staffNo, cnic, name, address, dateJoined, deptNo)`

staffNo	cnic	name	deptNo
S014	35202-...	Dr. Amina Raza	D01
S027	37405-...	Bilal Ahmed (Nurse)	D01

Every Staff row carries exactly one deptNo, matching the “many staff, one department” shape of WorksIn directly — the same rule already used for PropertyForRent.branchNo.

Rule 5 — Mapping M:N Relationships

An M:N relationship ALWAYS requires a brand-new relation — neither side's primary key alone can identify one occurrence of the relationship.



NurseWardAssignment

staffNo	wardNo
S027	W1
S027	W2
S041	W1

CREATE TABLE NurseWardAssignment

```
CREATE TABLE NurseWardAssignment (  
  staffNo VARCHAR(6) NOT NULL,  
  wardNo VARCHAR(4) NOT NULL,  
  PRIMARY KEY (staffNo, wardNo),  
  FOREIGN KEY (staffNo) REFERENCES Staff(staffNo),  
  FOREIGN KEY (wardNo) REFERENCES Ward(wardNo)  
);
```

This is exactly why relationships with attributes became entities in Lecture 13: whether you call it “a relationship with attributes” or “the M:N relationship's own relation,” you land on the identical structure.

Rule 6 — Mapping Multi-Valued Attributes

A multi-valued attribute can never be stored as an ordinary column — it ALWAYS becomes its own relation, with a composite primary key of the owning entity's key plus the attribute itself.

PatientPhone (patientNo, phoneNumber)

patientNo	phoneNumber
P0142	0300-1234567
P0142	021-34567890
P0198	0333-9876543

Mirrors Rule 2's weak-entity pattern almost exactly

phoneNumber is, in effect, treated the same way a weak entity's partial key is: meaningless without patientNo beside it. Same composite-key shape, different motivating construct.

Rule 7 — Mapping Specialization: Three Strategies

Mapping stops being mechanical here — Connolly & Begg present three viable strategies; the right one depends on the disjoint/overlapping and total/partial constraints from Lecture 14.

Strategy	Works well when	Trade-off
A — subclass-only	Disjoint AND total (every occurrence is exactly one subclass)	Shared attributes duplicated across tables; “all staff” needs a UNION
B — superclass + subclasses	ANY combination of constraints — the general-purpose default	One extra join needed to retrieve a subclass occurrence's full attributes
C — single table + discriminator	Disjoint, small subclass-specific attribute sets	Many NULL columns as subclasses grow — “attribute overload” reappearing

Recall: Staff specialized into four roles — disjoint, total (Lecture 15)

Rule 7 — Strategies A and C, Worked

Strategy A — Doctor(**staffNo**, **cnic**, **name**, **licenseNo**, **consultationFee**)

staffNo	cnic	name	licenseNo	consultationFee
S014	35202-...	Dr. Amina Raza	PMC-4471	2500

Strategy C — Staff(**staffNo**, **cnic**, **name**, **staffType**, **licenseNo**, **shift**, **labSpecialty**, **officeRole**)

staffNo	name	staffType	licenseNo	shift
S014	Dr. Amina Raza	Doctor	PMC-4471	NULL
S027	Bilal Ahmed	Nurse	NULL	Night

Strategy A drops the superclass table; Strategy C folds everything into one wide table

Strategy A: every subclass relation carries the FULL inherited attribute set, duplicated. Strategy C: every subclass's attributes folded into one table, left NULL where they don't apply, plus a discriminator column (staffType) recording which subclass a row actually is.

Rule 7 — Strategy B: the Default, and the Hospital's Multi-Level Case

`Staff(staffNo, cnic, name, ...)`

staffNo	cnic	name
S014	35202-...	Dr. Amina Raza

`Doctor(staffNo, licenseNo, consultationFee)`

staffNo	licenseNo	consultationFee
S014	PMC-4471	2500

staffNo is PK AND FK back into Staff on the Doctor relation — shared attrs live once on the superclass; each subclass relation holds only its own extra attributes.

`Consultant(staffNo, yearsOfExperience, officeRoomNo)`

staffNo	years	officeRoomNo
S014	12	C-204

`Resident(staffNo, rotationEndDate, supervisingConsultantNo)`

staffNo	rotationEndDate	supervisingConsultantNo
S099	2027-01-15	S014

Strategy B is the only one that composes cleanly across multiple levels

For the hospital's two-level hierarchy (Staff → Doctor → {Consultant, Resident}), Strategy B works with no modification. Resident.supervisingConsultantNo is a foreign key into Consultant.staffNo — NOT into Staff.staffNo — because the requirement was specifically “supervised by one of the CONSULTANTS,” and Strategy B's per-level relations let the FK target exactly the right level.

Rule 8 — Mapping Categorization (Union Types)

Because the superclasses' primary keys aren't even the same type (cnic vs. bankCode vs. regNo), the category needs its OWN independent surrogate key, and each superclass gets an optional foreign key pointing at it.

RegisteredOwner(ownerNo, ownerSince)

ownerNo	ownerSince
RO-001	2019-03-01

Person(cnic, name, ownerNo)

cnic	name	ownerNo
35202-1234567-1	Hina Tariq	RO-001

Person.ownerNo is nullable

Most Person rows are not vehicle owners at all — mirroring exactly the “put the FK on the side that may or may not participate” logic from Rule 3's 1:1 mapping. Bank and Company each get the identical treatment: their own nullable ownerNo foreign key into RegisteredOwner.

Rule 9 — Mapping Aggregation

Aggregation needs NO new mapping machinery of its own — it reduces entirely to relations already produced by earlier rules. The aggregate becomes a relation per Rules 1–6; the outer relationship becomes one additional foreign key column.

CREATE TABLE Admission

```
CREATE TABLE Admission (  
  admissionNo      VARCHAR(8)  NOT NULL,  
  admissionDate    DATE        NOT NULL,  
  dischargeDate    DATE,  
  patientNo       VARCHAR(6)  NOT NULL,  
  wardNo          VARCHAR(4)  NOT NULL,  
  bedNo           SMALLINT    NOT NULL,  
  attendingDoctorNo VARCHAR(6)  NOT NULL,  
  PRIMARY KEY (admissionNo),  
  FOREIGN KEY (patientNo) REFERENCES Patient(patientNo),  
  FOREIGN KEY (wardNo, bedNo) REFERENCES Bed(wardNo, bedNo),  
  FOREIGN KEY (attendingDoctorNo) REFERENCES Doctor(staffNo)  
);
```

A genuinely reassuring result

attendingDoctorNo is a completely ordinary foreign key by the time mapping is done. Aggregation is a conceptual tool for thinking clearly while modeling — it leaves no special trace in the final relational schema beyond an ordinary FK.

Rule 10 — Mapping EER Constraints

Mandatory (1,...) → NOT NULL

branchNo NOT NULL enforces “must always belong to a branch” directly.

Disjoint + Strategy C → CHECK

```
CHECK (staffType IN  
('Doctor','Nurse','Technician','Administrator'))
```

Disjoint + Strategy B → no single declarative constraint

Nothing in plain SQL stops a staffNo from appearing in more than one subclass table — needs a trigger.

Minimum cardinality of “1 or more” is a real, common gap

A foreign key naturally enforces “at most one” on the referencing side and “referenced value must exist” on the target — but has NO built-in way to enforce “this referenced row must be pointed at by at least one row.” Nothing stops a Department from being inserted with zero Staff ever assigned to it, even if the business rule is “every department must have at least one staff member.” Enforcing a true minimum cardinality of (1,*) generally requires a trigger or application-level check — the relational model's declarative constraints are genuinely weaker here than the EER diagram's multiplicity notation promises.

Summary: Every EER Construct, Mapped

EER construct	Relational mapping
Strong entity type	One relation; identifier → primary key
Weak entity type	One relation; composite key = partial key + owner's entire key
1:1 relationship	FK on the mandatory-participation side; no new relation
1:N relationship	FK on the "many" side; no new relation
M:N relationship	New relation; PK = combination of both sides' keys
Multi-valued attribute	New relation; PK = owner's key + the attribute
Relationship with attributes	New relation exactly as the M:N rule, holding the extra attributes
Specialization / generalization	Strategy A, B, or C – chosen from disjoint/total constraints
Aggregation	Aggregate maps via the rules above; outer relationship → one ordinary FK
Composition	Same as 1:N, plus ON DELETE CASCADE on the FK
Categorization / union type	New relation with surrogate key; each superclass gets a nullable FK

Key Takeaways

- Mapping proceeds in a fixed order — strong entities, weak entities, relationships, multi-valued attributes, specialization, categorization/aggregation, then constraints.
- 1:1 and 1:N relationships never need a new relation, only correct FK placement; M:N relationships and multi-valued attributes always need one.
- Specialization has three mapping strategies with genuinely different trade-offs — Strategy B (superclass + one relation per subclass) is the safest default and the only one that composes cleanly across multi-level hierarchies.
- Categorization needs its own surrogate key because its superclasses don't share compatible primary keys.
- Aggregation and composition both resolve entirely into ordinary relations and foreign keys once mapped — and not every EER constraint survives the trip to SQL with full declarative strength.